

A RISC-V Neuromorphic Micro-Controller Unit (vMCU) with Event-Based Physical Interface and Computational Memory for Low-Latency Machine Perception and Intelligence at the Edge

Daniel R. Mendat*, Jonah P. Sengupta*, Gaspar Tognetti*, Martin Villemur*,
Philippe O. Pouliquen*, Sergio Montano[†], Kayode Sanni[‡],
Jamal L. Molin[‡], Nishant Zachariah[‡], Isidoros Doxas[‡], Andreas G. Andreou*

* Department of Electrical & Computer Engineering, Johns Hopkins University

[†] Embedded Systems Division, Silicon Austria Labs

[‡] Mission Systems, Northrop Grumman Corporation

Email: {dmendat4, jsengup1, gtognet1, philippe, andreou} @jhu.edu, {martin.villemur}@silicon-austria.com

Abstract—Neuromorphic TinyML (vTinyML) aims at solving problems in machine perception and intelligence at the edge that necessitate low latency processing, using low resource processors that have neuromorphic event-based sensory interfaces and neuromorphic accelerators. In this paper, we report on a neuromorphic TinyML architecture (vMCU) which can be leveraged to be deployed to process data from event-based sensors on the edge. The core of the system is a RISC-V CPU from SiFive which is used for algorithm development. The CPU interfaces with a set of communication and computation peripherals which are comprised most notably of an event-based physical interface which has a 256Kib FIFO, a programmable 47-bit time-stamping unit and an embedded Compute in Memory (CiM) associative processor employing charge based processing and a pseudo-DRAM cell primitive. The vMCU SOC was fabricated in 65nm CMOS, has a die size of 7mm x 4mm, runs at 100MHz and has a maximum event throughput at its physical interface of 17Meps. Binary and integer operations on long bit vectors using the CiM accelerator capabilities take a few fJ per Op. vMCU capability consumes 30mW and is demonstrated in various tasks for embedded applications, including character recognition from a DAVIS240C event-based camera.

Index Terms—edge computing, SoC, compute-in-memory, neuromorphic, tinyML

I. INTRODUCTION

Applications such as machine-human interaction, robotics and virtual reality are challenging for modern state of the art embedded devices. Solutions to these issues can be gleaned from the way the brain goes about processing, as event-based communication and computation is deployed near the sensory origin. Neuromorphic TinyML seeks to realize microarchitectures which perform machine learning and computer vision tasks by leveraging sparse, event-based sensory information in a low-latency, energy-efficient manner.

Neuromorphic embedded platforms for architecture and algorithm exploration have been deployed on FPGAs and other microcontroller based platforms [1], [2]. However, a devoted neuromorphic microcontroller architecture with embedded computational and communication units has not been

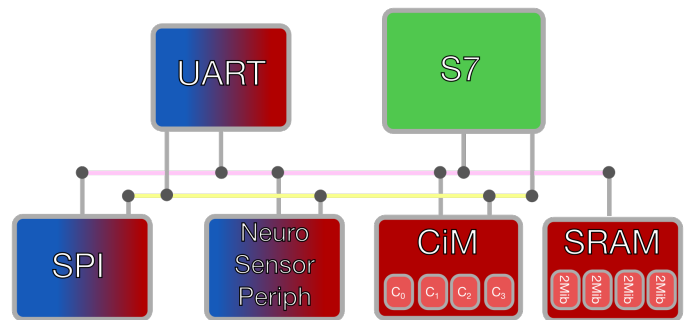


Fig. 1. vMCU system architecture block diagram: greed, red, and blue blocks correspond to bus controllers, peripherals and external interface. The pink bus denotes AHB connections while the yellow outlines interrupt connectivity.

reported. In this paper, we report on the architecture, fabrication, testing and application development of a RISC-V neuromorphic Microcontroller Unit (vMCU) with physical interfaces and micro-architectural units designed for low-latency embedded applications that employ event-based sensors (vision [3], audition [4], touch [5], etc.). Section II gives an overview of the vMCU architecture and RISC-V CPU core. Section III outlines the operation of the event-based sensor interface and compute-in-memory (CiM) accelerator. Finally, the vMCU operation is demonstrated and preliminary specifications are outlined in Section IV before concluding in Section V.

II. vMCU ARCHITECTURE

An overall block diagram of the embedded SoC ASIC is seen in Figure 1. The microarchitecture consists of a series of computational and communication controllers and peripherals that all interact over a 64-bit Advanced Microcontroller Bus Architecture (AMBA) [6] Advanced High-performance Bus (AHB) matrix. At the heart of the system is a SiFive S7 series core (RV6IMAFDS) with single and double precision floating

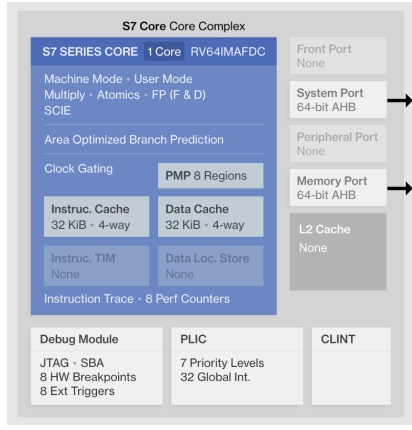


Fig. 2. SiFive S7 Series RISC-V CPU Configuration Details

point units, 32KiB instruction and data cache, 64-bit AHB system/memory ports, standard UART and SPI interfaces as well as four banks of 2MiB SRAM (1MiB total). External communication is established through a pairing of UART and SPI interfaces that load S7 programs and realize serial data transfer. System memory is composed of a 1 MiB SRAM module that houses instruction space and parameter data. Two additional peripherals were leveraged to enable efficient, neuromorphic perception on the edge. The first is the Neuromorphic Sensor Peripheral (NSP) which receives address events (AEs) from an off-chip, event-based camera, similar to [3]. This module is responsible for decoding events and communicating to the CPU via interrupt signaling to read and process the data. Finally, a quad-core **CiM** accelerator was designed and inserted to efficiently compute fixed-point arithmetic, requisite for most machine learning and computer vision applications, in an energy efficient manner.

A. SiFive CPU

The SiFive S7 series RISC-V CPU [7] is the main controller for the embedded algorithms and the peripherals in the SoC. As seen in Figure 2, the configuration includes a 64-bit double precision floating-point core, 32KiB 4-way instruction and data caches, and 32 global interrupts with 7 priority levels handled by the Platform-Level Interrupt Controller (PLIC). There is also the option of using a Core Local Interrupter (CLINT). Memory was attached to the 64-bit AHB Memory Port so that the caches could be utilized, and other peripherals, including the CiM accelerator and the NSP, were attached to the System Port. A 1 MiB SRAM was loaded with the compiled software via pre-programmed bootloader and UART or SPI peripherals. The prebuilt RISC-V GCC toolchain is used to program the system, and it is complemented with custom Application Programming Interfaces (APIs) for the specialized interface and processing units. These custom APIs abstract away some of the details required for using the interface and processing units, including reading from and writing to the proper memory-mapped addresses for setting

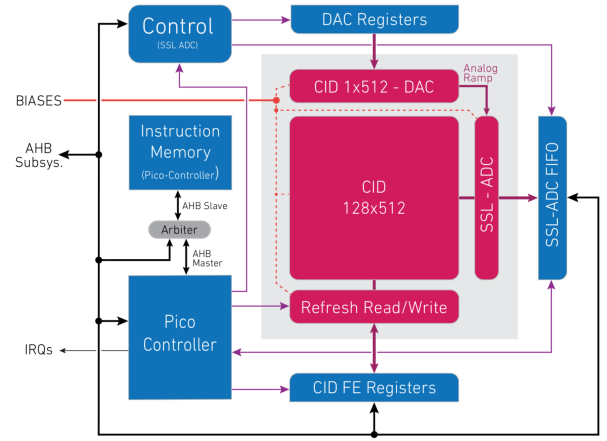


Fig. 3. CiM mixed-signal subsystem: Blue blocks were implemented in the digital domain while red-pink blocks are analog designs.

configuration parameters, kicking off the peripherals' capabilities, and reading out results, among other tasks.

III. NEUROMORPHIC COMMUNICATION AND COMPUTATION PERIPHERALS

Computation and communication peripherals were added to the SoC to allow for programmability, sensor connectivity, and energy-efficient, fixed-point arithmetic. SPI and UART peripherals were used to migrate data on and off of the chip including during CPU boot. Three external pins configured the bootloader to read from either peripheral for initial program configuration while also selecting the target destination in system memory. Ultimately, these can also be configured to work with the bevy of readily available UART or SPI wireless modules to enable the SoC for IoT applications.

A. Compute-in-Memory Accelerator

Our mixed-signal, CiM-based subsystem is shown in Figure 3. Peripheral parameters, instructions, and data are configured by the S7 CPU via AHB bus. A custom, pipelined, reduced-instruction set microcontroller was implemented adjacent to the accelerator to allow for a programmable solution to the complex signaling needed by the analog block and to shuttle data in the subsystem. Another controller was implemented to provide the necessary signaling for the analog-to-digital conversion of the computation results. Finally, these digital results are stored in an output FIFO that is ultimately read from using the S7 CPU. The computational element of the CiM accelerator is a 2.5T pseudo-DRAM bit-cell, otherwise known as a charge-injection device or CID [8]. An associated crossbar with these elements is seen in Figure 4. For programming, the bit-cell is selected via asserted word-line and the data is presented on the bit-line. Peripheral column-wise sense amplifiers then detect these programmed values and refresh them periodically. These represent the weights or $w_{i,j}$ values for the operation. For a computation, data, x_j , is presented on the gate of the half-transistor. If $w_{i,j} = 1$ and $x_j = 1$, the gate of the center device, or the math-line, is

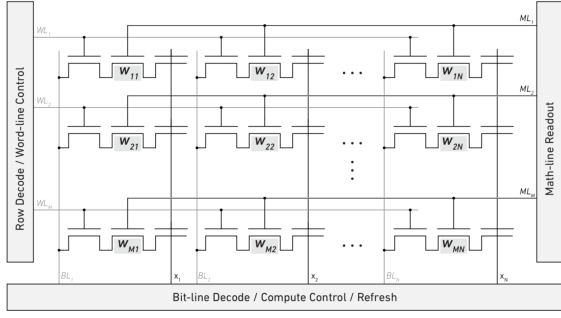


Fig. 4. CiM analog crossbar for charge-mode computation

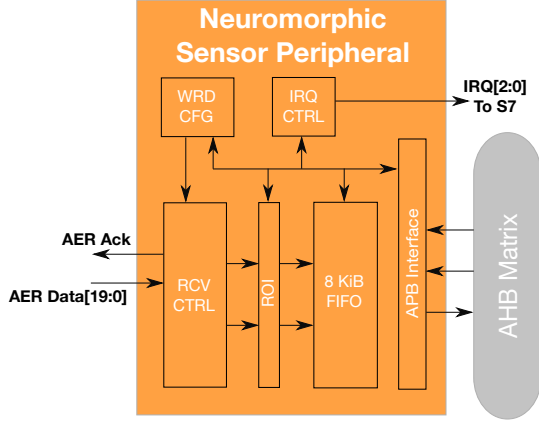


Fig. 5. Neuromorphic Sensor Peripheral connectivity and sub-modules

lowered by ΔV_i , hence implementing a charge mode, 1-bit AND. By using a common math-line, it is shown that

$$\Delta V_i = \frac{\alpha}{N} \Delta V_{BL} \sum_{j=0}^{N-1} w_{i,j} x_j \quad (1)$$

where α depends on the oxide and junction capacitance of the middle device and ΔV_{BL} is the change in bit-line potential above the threshold of the half-device. This operation can then be abstracted as a vector-vector multiplication (dot-product) or when computing across multiple rows, a vector-matrix multiplication:

$$Z_i = \sum_{j=0}^{N-1} w_{i,j} x_j = W_j \cdot X \quad (2)$$

$$Z = XW \quad (3)$$

More detail about the charge mode operation of the crossbar can be found in [8]–[10]. The output of the follower is routed to a comparator which is leveraged to convert the analog potential to a digital word via single-slope, analog-to-digital conversion (SSL-ADC). Comparator outputs are then used to latch a synchronized count value to complete the conversion.

B. Neuromorphic Sensor Peripheral

A block diagram of the NSP can be seen in Figure 5. The purpose of the sensor peripheral is to decode data from the

off-chip neuromorphic sensor, buffer resulting address events (AE) in an intermediate buffer stage, and coordinate intra-chip transfers by utilizing interrupts to signal the S7.

All event-based data transmission into the SoC was accomplished via a four-phase, asynchronous, delay-insensitive interface from an off-chip neuromorphic vision sensor [11]. Each AE conveys the location of a triggered event from the event-based camera. In a burst-column approach, the row address is transmitted before sending the column addresses of triggered pixels from that row. The interface forgoes using a request signal to circumvent delay with respect to the data bus. Hence, data is encoded in a one-hot fashion where the 0 state designates an active low state.

Data is then decoded into 8-bit row and column addresses within the system peripheral and stored in the local 32 KiB event FIFO. Each AE is tagged with a 47-bit timestamp whose resolution is configurable via the Advanced Peripheral Bus (APB) interface. A polarity bit to designate ON and OFF events is appended to each AE and comprises the 64-bit word $e_i = \{x_i, y_i, p_i, ts_i\}$, where x_i and y_i are the x and y coordinates for the event e_i , p_i is the event polarity, and ts_i is the timestamp. In addition to customizable data storage, the peripheral can be configured to only store events in a certain rectangular region-of-interest (ROI). AE address and timestamp resolutions can also be subsampled by a value set by the S7. By subsampling, the timestamp precision decreases while AE throughput increases as more AEs are placed on the AMBA bus.

The peripheral also dictates the event-based processing flow within the SoC via assertion of three configurable CPU interrupt requests (IRQs) via the IRQ CFG block. Two interrupts are the full and empty signals of the internal FIFO buffer. A final interrupt is a flag asserted when N_{thresh} events are accumulated in the FIFO. N_{thresh} is a 32-bit value configurable by the S7. Each interrupt can be masked by memory-mapped registers.

IV. RESULTS

The vMCU architecture was designed, verified, and implemented in a low-power 65nm CMOS technology. Top layout of the design with module labeling is shown in Figure 6. The majority of the ASIC footprint is consumed by the 1MiB of system memory. The quad-core CiM accelerator includes 32KiB of pseudo-DRAM which can be utilized for additional storage or leveraged for the energy-efficient arithmetic described in Section III-A. The NSP utilized a 4096x64 (32KiB) FIFO for internal event buffering. To verify and characterize the SoC, an Opal Kelly XEM7310 FPGA Development board and Python API was utilized to configure on-board DACs for chip supplies and biases and send serial data to the chip UART and SPI interfaces. The test set-up is seen in Figure 7.

As a preliminary test and to show the embedded processing potential, a quantized version of the LeNet neural network was deployed on the S7 CPU to perform edge inference on MNIST digits [12]. During operation, the SoC was measured to dissipate 30mW of power which is a factor of 30 less

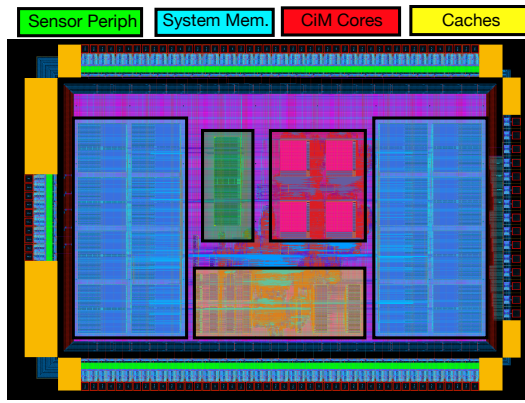


Fig. 6. Top level layout of the fabricated design in a 65nm low-power technology. Submodules of the SoC are highlighted and designated accordingly.

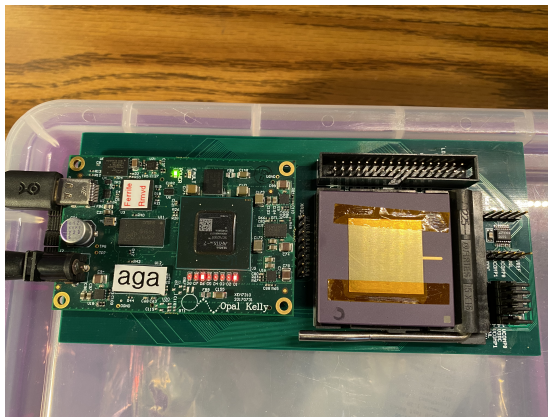


Fig. 7. Test board with SoC ASIC: Opal Kelly XEM7310 FPGA on board with packaged test chip.

than that seen on a comparable architecture deployed on an FPGA [13]. Data from the N-MNIST dataset was generated by saccading a DAVIS240C camera in front of MNIST samples [14]. Image data from the test samples was transmitted to the vMCU using AER transmitter model outlined in prior work [13]. This module is implemented on an Opal Kelly XEM6310 FPGA development board. Each sample is thresholded using the median value of the image in order to produce a binary representation. Next, valid pixel addresses are one-hot encoded, transmitted to the S7 SoC and received by SensorPeripheral. After a certain amount of events have been accumulated in the output FIFO, the S7 reads from the peripheral and populates an internal array with the decoded image data. Following the readout, the S7 does a forward pass through the embedded neural network using the received image, and the classification result is transmitted to the PC via the UART serial interface. A confusion matrix acquired from validating the test dataset on the re-trained model is seen in Figure 8. A 95% classification accuracy was achieved using the embedded model.

Additional work is needed to fully incorporate the CiM accelerator for the neural network multiply and accumulate operations. Individual characterization of the CiM accelerator

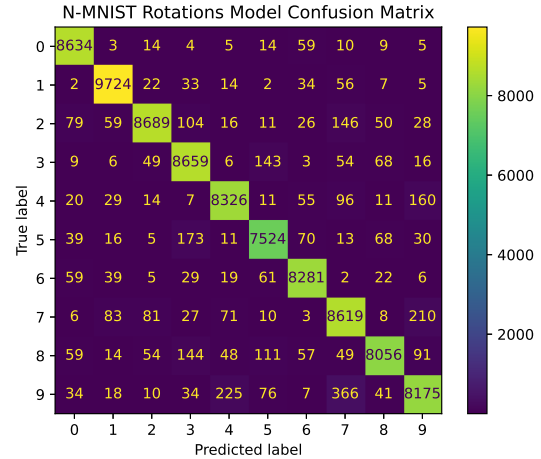


Fig. 8. Confusion Matrix yielded from LeNet implemented on the vMCU utilizing the NSP for image data communication - N-MNIST

TABLE I
ASIC SoC PERFORMANCE

Metric	Value
Clock Speed	100 MHz
Die Dimensions	7mm x 4mm
Max Event Readout	17Meps
CiM Cell Density	1.55M/mm ²
1-bit MAC Throughput	2.1 * 10 ¹¹ Op/s
1-bit efficiency	45fJ/Op
8-bit MAC Throughput	3.3 * 10 ⁹ Op/s
8-bit efficiency	1.47pJ/Op
Power Consumption	30 mW

was conducted to resolve the computational efficiency of the peripheral. At a 100 MHz system clock, two operations were used: a 1-bit multiply and accumulate (single threshold), similar to a neuron activation function, and a full 8-bit, multiply and accumulate. The recorded throughput, included overhead for data shuttling, is reflected in Table I along with other pertinent metrics.

V. CONCLUSION

A low-power, mixed-signal TinyML SoC for energy-efficient, event-based perception on the edge was outlined and demonstrated. System algorithms were compiled and run on a SiFive S7 CPU core with integrated UART and SPI serial interfaces for configuration and communication to off-chip devices. A Neuromorphic Sensor Peripheral was designed to communicate with a board-adjacent event-based sensor and prompt CPU processing. Energy-efficient, fixed point arithmetic was performed on a mixed-signal, Compute-in-Memory peripheral which can perform 1-bit MACs at 45fJ/Op and 8-bit operations at 1.47pJ/Op. Leveraging data acquired from a neuromorphic sensor and a quantized neural network model, it was shown to perform a character recognition task while consuming 30mW. Further work is needed to fully integrate the CiM accelerator into the hardware and software stack, but

the vMCU has shown promise to be the hardware basis of future development in neuromorphic TinyML.

ACKNOWLEDGMENT

This work was funded by the Defense Advanced Research Projects Agency (DARPA) Microsystems Technology Office (MTO) ReImagine project under Contract HR0011-17-C0071 and in part by the Northrop Grumman Faculty Award to support graduate student research (GT).

REFERENCES

- [1] J. P. Sengupta, M. Villemur, P. O. Pouliquen, P. Julian, and A. G. Andreou, "Embedded processing pipeline exploration for neuromorphic event based perceptual systems," *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*, p. 1–5, May 2022.
- [2] M. Hofstätter, P. Schön, and C. Posch, "A sparcc-compatible general purpose address-event processor with 20-bit 10ns-resolution asynchronous sensor data interface in 0.18 μ m cmos," in *Proceedings of 2010 IEEE International Symposium on Circuits and Systems*, 2010, pp. 4229–4232.
- [3] P. Lichtsteiner, C. Posch, and T. Delbruck, "A 128 $\times$ 128 120 db 15 μ s latency asynchronous temporal contrast vision sensor," *IEEE Journal of Solid-State Circuits*, vol. 43, no. 2, pp. 566–576, 2008.
- [4] M. Yang, C.-H. Chien, T. Delbruck, and S.-C. Liu, "A 0.5 v 55uw 64x2 channel binaural silicon cochlea for event-driven stereo-audio sensing," *IEEE Journal of Solid-State Circuits*, vol. 51, no. 11, pp. 2554–2569, 2016.
- [5] W. W. Lee, S. L. Kukreja, and N. V. Thakor, "A kilohertz kilotaxel tactile sensor array for investigating spatiotemporal features in neuromorphic touch," in *2015 IEEE Biomedical Circuits and Systems Conference (BioCAS)*. IEEE, 2015, pp. 1–4.
- [6] "AMBA Specification (Rev 2.0)," <https://developer.arm.com/documentation/ih0011/a>, accessed: 2020-10-21.
- [7] "SiFive S76 Standard Core," <https://www.sifive.com/cores/s76>, accessed: 2020-10-21.
- [8] R. Karakiewicz, R. Genov, and G. Cauwenberghs, "1.1 tmacs/mw fine-grained stochastic resonant charge-recycling array processor," *Sensors Journal, IEEE*, vol. 12, no. 4, pp. 785–792, 2012.
- [9] G. Tognetti, J. Sengupta, P. O. Pouliquen, and A. G. Andreou, "Characterization of a pseudo-dram crossbar computational memory array in 55nm cmos : (invited paper)," in *2019 53rd Annual Conference on Information Sciences and Systems (CISS)*, 2019, pp. 1–5.
- [10] G. Tognetti, "Circuits and architecture for bio-inspired ai accelerators," Ph.D. dissertation, The Johns Hopkins University, 2020.
- [11] J. Lin and K. Boahen, "A delay-insensitive address-event link," in *2009 15th IEEE Symposium on Asynchronous Circuits and Systems*. IEEE, 2009, pp. 55–62.
- [12] H. Unlu, "Efficient neural network deployment for microcontroller," *arXiv preprint arXiv:2007.01348*, 2020.
- [13] J. P. Sengupta, M. Villemur, D. R. Mendat, G. Tognetti, and A. G. Andreou, "Architecture and algorithm co-design framework for embedded processors in event-based cameras," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2021, pp. 1–5.
- [14] G. Orchard, A. Jayawant, G. K. Cohen, and N. Thakor, "Converting static image datasets to spiking neuromorphic datasets using saccades," *Frontiers in neuroscience*, vol. 9, p. 437, 2015.